Lecture 6: Data Wrangling

Heather Mattie, PhD

September 18, 2025

Recipes of the Day!

Happy Hispanic Heritage Month!

Beef Empanadas

Announcements

- There will be lab this week
- Homework #1 deadline pushed to Friday, September 26
- Grab a duckie!

Data Wrangling

Data Wrangling Roadmap

1. `dplyr` functions
2. Importing data
3. Reshaping data
4. Combining data frames
5. Parsing dates and times

Data Wrangling

`dplyr` functions

Heather Mattie, PhD

HARVARD T.H. CHAN
SCHOOL OF PUBLIC HEALTH

The Data Science Pipeline

Collecting and importing (loading) the data

Processing (cleaning, wrangling) the data

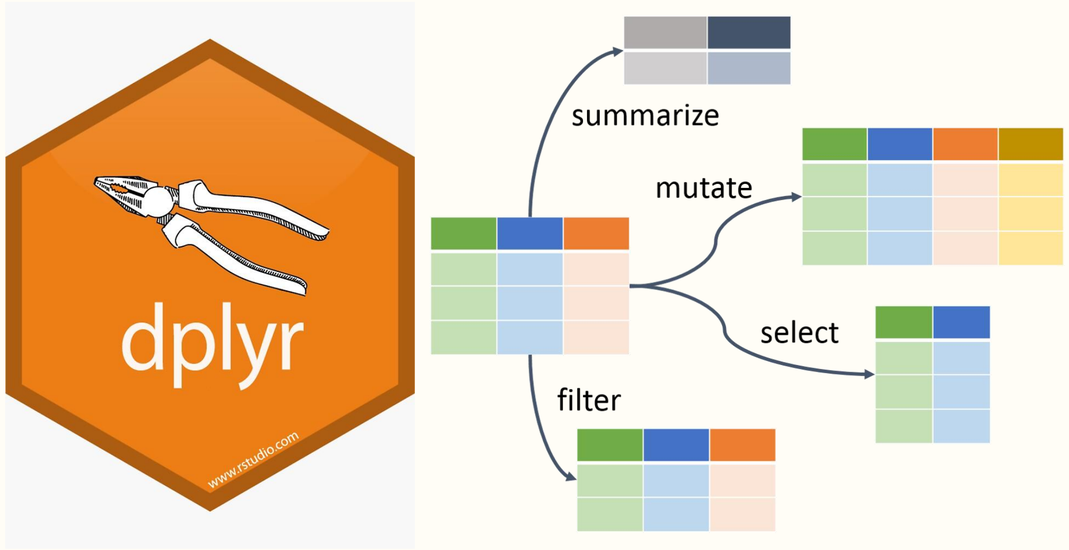
Visualizing and summarizing the data

Building models (statistical and ML)

Interpretation and communication of results

The `dplyr` package

`dplyr` is an R package containing functions that are useful for manipulating data frames in an intuitive, user-friendly way



The pipe

The pipe operator, `%>%`, is a powerful tool for chaining multiple operations (coding steps) together

- Code is more readable
- Code is more efficient
- Takes the output of one function and uses it as the input of the next function or expression

mutate

The `mutate` function creates new columns

- Can be functions of existing variables
- Can modify and delete existing columns

```
library(dslabs)
data(murders)
head(murders)
```

```
##        state abb region population total
## 1    Alabama  AL  South    4779736   135
## 2     Alaska  AK   West     710231    19
## 3    Arizona  AZ   West    6392017   232
## 4   Arkansas  AR  South    2915918    93
## 5 California  CA   West   37253956  1257
## 6   Colorado  CO   West    5029196    65
```

```
library(dplyr)
murders <- murders %>% mutate(murder_rate = total / population * 100000)
head(murders)
```

```
##        state abb region population total murder_rate
## 1    Alabama  AL  South    4779736   135    2.824424
## 2     Alaska  AK   West     710231    19    2.675186
## 3    Arizona  AZ   West    6392017   232    3.629527
## 4   Arkansas  AR  South    2915918    93    3.189390
## 5 California  CA   West   37253956  1257    3.374138
## 6   Colorado  CO   West    5029196    65    1.292453
```

filter

The `filter` function is used to subset a data frame

- Retains rows that satisfy all conditions
- When a condition results in NA, the row will be dropped

```
murders %>% filter(murder_rate <= 0.71)
```

```
##           state abb        region population total murder_rate
## 1        Hawaii  HI          West    1360301     7   0.5145920
## 2          Iowa  IA North Central    3046355    21   0.6893484
## 3 New Hampshire  NH     Northeast    1316470     5   0.3798036
## 4  North Dakota  ND North Central     672591     4   0.5947151
## 5       Vermont  VT     Northeast     625741     2   0.3196211
```

select

The `select` function is used to select variables (columns) based on their name

- Can also be used to rename columns

```
murders %>% select(state, region, murder_rate) %>%
filter(murder_rate <= 0.71)
```

```
##           state        region murder_rate
## 1        Hawaii          West   0.5145920
## 2          Iowa North Central   0.6893484
## 3 New Hampshire     Northeast   0.3798036
## 4  North Dakota North Central   0.5947151
## 5       Vermont     Northeast   0.3196211
```

summarize

The `summarize` function is used to create summary statistics of data

- Takes a data frame and outputs a new data frame with a single row containing specified summary calculations
- Returns a new **data frame** with one row for each group if the original data frame has been grouped by columns and/or rows

```
us_murder_rate <- murders %>%
  summarize(murder_rate = sum(total) / sum(population) * 100000)

us_murder_rate
```

```
##   murder_rate
## 1    3.034555
```

```
us_murder_rate <- murders %>%
  summarize( murder_rate = sum(total) / sum(population) * 100000) %>%
  .$murder_rate

us_murder_rate
```

```
## [1] 3.034555
```

group_by

The `group_by` function takes a data frame and converts it into a grouped data frame where operations are performed by group

- `ungroup()` removes the grouping

```
murders %>%
  group_by(region) %>%
  summarize(median_rate = median(murder_rate))
```

```
## # A tibble: 4 × 2
##   region        median_rate
##   <fct>               <dbl>
## 1 Northeast            1.80
## 2 South                3.40
## 3 North Central        1.97
## 4 West                 1.29
```

arrange

The **arrange** function orders the rows of a data frame by the values of selected columns

- The default is ascending order

```
murders %>%
  arrange(murder_rate) %>%
  head()
```

```
##           state abb        region population total murder_rate
## 1       Vermont  VT     Northeast     625741     2   0.3196211
## 2 New Hampshire  NH     Northeast    1316470     5   0.3798036
## 3        Hawaii  HI          West    1360301     7   0.5145920
## 4  North Dakota  ND North Central     672591     4   0.5947151
## 5          Iowa  IA North Central    3046355    21   0.6893484
## 6         Idaho  ID          West    1567582    12   0.7655102
```

arrange

The `arrange` function orders the rows of a data frame by the values of selected columns

- The default is ascending order
- Can use the `desc()` function to switch to descending order

```
murders %>%
  arrange(desc(murder_rate)) %>%
  head()
```

```
##                  state abb        region population total murder_rate
## 1 District of Columbia  DC         South     601723    99   16.452753
## 2            Louisiana  LA         South    4533372   351    7.742581
## 3             Missouri  MO North Central    5988927   321    5.359892
## 4             Maryland  MD         South    5773552   293    5.074866
## 5       South Carolina  SC         South    4625364   207    4.475323
## 6             Delaware  DE         South     897934    38    4.231937
```

slice

The `slice` set of functions index rows by their integer locations

- Used to select, remove, and duplicate rows

```
murders %>% slice(1:10)
```

```
##                   state abb    region population total murder_rate
## 1               Alabama  AL     South    4779736   135    2.824424
## 2                Alaska  AK      West     710231    19    2.675186
## 3               Arizona  AZ      West    6392017   232    3.629527
## 4              Arkansas  AR     South    2915918    93    3.189390
## 5            California  CA      West   37253956  1257    3.374138
## 6              Colorado  CO      West    5029196    65    1.292453
## 7           Connecticut  CT Northeast    3574097    97    2.713972
## 8              Delaware  DE     South     897934    38    4.231937
## 9  District of Columbia  DC     South     601723    99   16.452753
## 10              Florida  FL     South   19687653   669    3.398069
```

slice

The `slice` set of functions index rows by their integer locations

- Used to select, remove, and duplicate rows

```
murders %>% slice(-c(3:5, 9:51))
```

```
##         state abb    region population total murder_rate
## 1     Alabama  AL     South    4779736   135    2.824424
## 2      Alaska  AK      West     710231    19    2.675186
## 3    Colorado  CO      West    5029196    65    1.292453
## 4 Connecticut  CT Northeast    3574097    97    2.713972
## 5    Delaware  DE     South     897934    38    4.231937
```

slice

The `slice` set of functions index rows by their integer locations

- Used to select, remove, and duplicate rows

```
murders %>% slice_head(n = 3)
```

```
##     state abb region population total murder_rate
## 1 Alabama  AL  South    4779736   135    2.824424
## 2  Alaska  AK   West     710231    19    2.675186
## 3 Arizona  AZ   West    6392017   232    3.629527
```

```
murders %>% slice_tail(n = 5)
```

```
##           state abb        region population total murder_rate
## 1      Virginia  VA         South    8001024   250   3.1246001
## 2    Washington  WA          West    6724540    93   1.3829942
## 3 West Virginia  WV         South    1852994    27   1.4571013
## 4     Wisconsin  WI North Central    5686986    97   1.7056487
## 5       Wyoming  WY          West     563626     5   0.8871131
```

Summary

- The `dplyr` package contains functions that are useful in wrangling data in a data frame
- The pipe operator is a tool to chain multiple operations together in an efficient way
- Throughout this course we will be using the `mutate`, `filter`, `select`, `summarize`, `group_by`, `arrange`, and `slice` functions to manipulate data frames

Data Wrangling

Importing data

Heather Mattie, PhD

HARVARD T.H. CHAN
SCHOOL OF PUBLIC HEALTH

Tidy data

Tidy data is a standardized way to structure datasets so they are easy to understand and analyze

- Each observation is a row
- Each variable is a column
- The `tidyverse` package can be used for data wrangling

Gapminder example

Tidy data format

```
library(tidyverse)
library(dslabs)
data("gapminder")
head(gapminder)
```

```
##                country year infant_mortality life_expectancy fertility
## 1              Albania 1960           115.40           62.87      6.19
## 2              Algeria 1960           148.20           47.50      7.65
## 3               Angola 1960           208.00           35.98      7.32
## 4 Antigua and Barbuda 1960               NA           62.97      4.43
## 5           Argentina 1960            59.87           65.39      3.11
## 6             Armenia 1960               NA           66.86      4.55
##   population          gdp continent          region
## 1    1636054           NA    Europe Southern Europe
## 2   11124892  13828152297    Africa Northern Africa
## 3    5270844           NA    Africa   Middle Africa
## 4      54681           NA  Americas       Caribbean
## 5   20619075 108322326649  Americas   South America
## 6    1867396           NA      Asia    Western Asia
```

Gapminder example

Raw data

```
library(tidyverse)
library(dslabs)
filename <- "https://raw.githubusercontent.com/rafalab/dslabs/master/inst/extdata/fertility-two-countries-example.csv"
wide_data <- read_csv(filename)
head(wide_data)
```

```
## # A tibble: 2 × 57
##   country  `1960` `1961` `1962` `1963` `1964` `1965` `1966` `1967` `1968` `1969`
##   <chr>     <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1 Germany    2.41   2.44   2.47   2.49   2.49   2.48   2.44   2.37   2.28   2.17
## 2 South K…   6.16   5.99   5.79   5.57   5.36   5.16   4.99   4.85   4.73   4.62
## # ℹ 46 more variables: `1970` <dbl>, `1971` <dbl>, `1972` <dbl>, `1973` <dbl>,
## #   `1974` <dbl>, `1975` <dbl>, `1976` <dbl>, `1977` <dbl>, `1978` <dbl>,
## #   `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>, `1983` <dbl>,
## #   `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
## #   `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>,
## #   `1994` <dbl>, `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>,
## #   `1999` <dbl>, `2000` <dbl>, `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, …
```

Working directory

The working directory is where R will save or look for files by default

- When importing data from a file on a computer, R will search the files in the working directory first

```
getwd()
```

```
## [1] "/Users/hem122/Desktop"
```

Paths

If the file is not in the working directory, a path to the file is required to import it

- The path indicates the location (folder) of the file on a computer or on a website

```
path <- system.file("extdata", package = "dslabs")
path
```

```
## [1] "/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/
library/dslabs/extdata"
```

```
filename <- "murders.csv"
fullpath <- file.path(path, filename)
fullpath
```

```
## [1] "/Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/
library/dslabs/extdata/murders.csv"
```

readr

The **readr** package includes functions for reading data stored in a spreadsheet file into R

Function	Format	Typical suffix
read_table	white space separated values	txt
read_csv	comma separated values	csv
read_csv2	semicolon separated values	csv
read_tsv	tab delimited separated values	tsv
read_delim	general text file format, must define delimiter	txt

`readxl`

The **`readxl`** package provides functions for reading in Microsoft Excel formats

Function	Format	Typical suffix
read_excel	auto detect the format	xls, xlsx
read_xls	original format	xls
read_xlsx	new format	xlsx

Raw Gapminder data

```
library(tidyverse)
library(dslabs)
filename <- "https://raw.githubusercontent.com/rafalab/dslabs/master/inst/extdata/fertility-two-countries-example.csv"
wide_data <- read_csv(filename)
head(wide_data)
```

```
## # A tibble: 2 × 57
##   country  `1960` `1961` `1962` `1963` `1964` `1965` `1966` `1967` `1968` `1969`
##   <chr>     <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1 Germany    2.41   2.44   2.47   2.49   2.49   2.48   2.44   2.37   2.28   2.17
## 2 South K…   6.16   5.99   5.79   5.57   5.36   5.16   4.99   4.85   4.73   4.62
## # ℹ 46 more variables: `1970` <dbl>, `1971` <dbl>, `1972` <dbl>, `1973` <dbl>,
## #   `1974` <dbl>, `1975` <dbl>, `1976` <dbl>, `1977` <dbl>, `1978` <dbl>,
## #   `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>, `1983` <dbl>,
## #   `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
## #   `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>,
## #   `1994` <dbl>, `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>,
## #   `1999` <dbl>, `2000` <dbl>, `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, …
```

Summary

- Data wrangling is the process of transforming raw data into a useable format called tidy data
- Tidy data format requires each variable to be a column and each observation a row
- The **`readr`** and **`readxl`** packages contain functions to read in spreadsheets of data

Data Wrangling

Reshaping data

Heather Mattie, PhD

HARVARD T.H. CHAN
SCHOOL OF PUBLIC HEALTH

Raw Gapminder data

```
library(tidyverse)
library(dslabs)
filename <- "https://raw.githubusercontent.com/rafalab/dslabs/master/inst/extdata/fertility-two-countries-example.csv"
wide_data <- read_csv(filename)
head(wide_data)
```

```
## # A tibble: 2 × 57
##   country  `1960` `1961` `1962` `1963` `1964` `1965` `1966` `1967` `1968` `1969`
##   <chr>     <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1 Germany    2.41   2.44   2.47   2.49   2.49   2.48   2.44   2.37   2.28   2.17
## 2 South K…   6.16   5.99   5.79   5.57   5.36   5.16   4.99   4.85   4.73   4.62
## # ℹ 46 more variables: `1970` <dbl>, `1971` <dbl>, `1972` <dbl>, `1973` <dbl>,
## #   `1974` <dbl>, `1975` <dbl>, `1976` <dbl>, `1977` <dbl>, `1978` <dbl>,
## #   `1979` <dbl>, `1980` <dbl>, `1981` <dbl>, `1982` <dbl>, `1983` <dbl>,
## #   `1984` <dbl>, `1985` <dbl>, `1986` <dbl>, `1987` <dbl>, `1988` <dbl>,
## #   `1989` <dbl>, `1990` <dbl>, `1991` <dbl>, `1992` <dbl>, `1993` <dbl>,
## #   `1994` <dbl>, `1995` <dbl>, `1996` <dbl>, `1997` <dbl>, `1998` <dbl>,
## #   `1999` <dbl>, `2000` <dbl>, `2001` <dbl>, `2002` <dbl>, `2003` <dbl>, …
```

pivot_longer

The `pivot_longer` function converts wide data into tidy data

- The first argument is which columns to collapse
- The second argument is the name of the column that will store the values of the collapsed columns
- The third argument is the name of the column that will store the values in the original data frame

```
new_tidy_data <- wide_data %>%
  pivot_longer(`1960`:`2015`, names_to = "year", values_to = "fertility")
head(new_tidy_data)
```

```
## # A tibble: 6 × 3
##   country year  fertility
##   <chr>   <chr>     <dbl>
## 1 Germany 1960       2.41
## 2 Germany 1961       2.44
## 3 Germany 1962       2.47
## 4 Germany 1963       2.49
## 5 Germany 1964       2.49
## 6 Germany 1965       2.48
```

pivot_longer

The `pivot_longer` function converts wide data into tidy data

- The first argument is which columns to collapse
- The second argument is the name of the column that will store the values of the collapsed columns
- The third argument is the name of the column that will store the values in the original data frame

```
new_tidy_data <- wide_data %>%
  pivot_longer(-country, names_to = "year", values_to = "fertility")
head(new_tidy_data)
```

```
## # A tibble: 6 × 3
##   country year  fertility
##   <chr>   <chr>     <dbl>
## 1 Germany 1960       2.41
## 2 Germany 1961       2.44
## 3 Germany 1962       2.47
## 4 Germany 1963       2.49
## 5 Germany 1964       2.49
## 6 Germany 1965       2.48
```

pivot_longer

The `pivot_longer` function converts wide data into tidy data

- The first argument is which columns to collapse
- The second argument is the name of the column that will store the values of the collapsed columns
- The third argument is the name of the column that will store the values in the original data frame

```
new_tidy_data <- wide_data %>%
  pivot_longer(-country, names_to = "year", values_to = "fertility") %>%
  mutate(year = as.integer(year))
head(new_tidy_data)
```

```
## # A tibble: 6 × 3
##   country  year fertility
##   <chr>   <int>     <dbl>
## 1 Germany  1960      2.41
## 2 Germany  1961      2.44
## 3 Germany  1962      2.47
## 4 Germany  1963      2.49
## 5 Germany  1964      2.49
## 6 Germany  1965      2.48
```

pivot_wider

The **pivot_wider** function converts tidy data into wide data

- The first argument, **names_to**, indicates which variable will be used as column names
- The second argument, **values_from**, specifies which variable to use to fill in the cells

```
new_wide_data <- new_tidy_data %>%
  pivot_wider(names_from = year, values_from = fertility)

new_wide_data %>% select(country, `1960`:`1967`)
```

```
## # A tibble: 2 × 9
##   country     `1960` `1961` `1962` `1963` `1964` `1965` `1966` `1967`
##   <chr>        <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1 Germany       2.41   2.44   2.47   2.49   2.49   2.48   2.44   2.37
## 2 South Korea   6.16   5.99   5.79   5.57   5.36   5.16   4.99   4.85
```

Raw Gapminder data #2

```
path     <- system.file("extdata", package = "dslabs")
filename <- file.path(path, "life-expectancy-and-fertility-two-countries-example.csv")

raw_data <- read_csv(filename)
select(raw_data, 1:5)
```

```
## # A tibble: 2 × 5
##   country     `1960_fertility` `1960_life_expectancy` `1961_fertility`
##   <chr>                  <dbl>                  <dbl>            <dbl>
## 1 Germany                 2.41                   69.3             2.44
## 2 South Korea             6.16                   53.0             5.99
## # ℹ 1 more variable: `1961_life_expectancy` <dbl>
```

Raw Gapminder data #2

```
data <- raw_data %>% pivot_longer(-country)
head(data)
```

```
## # A tibble: 6 × 3
##   country name                value
##   <chr>   <chr>               <dbl>
## 1 Germany 1960_fertility       2.41
## 2 Germany 1960_life_expectancy 69.3
## 3 Germany 1961_fertility       2.44
## 4 Germany 1961_life_expectancy 69.8
## 5 Germany 1962_fertility       2.47
## 6 Germany 1962_life_expectancy 70.0
```

separate

The **separate** function is used to split a single column into multiple columns

- The default character to split on is an underscore _

```
data %>% separate(name, c("year", "variable_name"), "_")
```

```
## # A tibble: 224 × 4
##    country year  variable_name value
##    <chr>   <chr> <chr>         <dbl>
##  1 Germany 1960  fertility      2.41
##  2 Germany 1960  life          69.3
##  3 Germany 1961  fertility      2.44
##  4 Germany 1961  life          69.8
##  5 Germany 1962  fertility      2.47
##  6 Germany 1962  life          70.0
##  7 Germany 1963  fertility      2.49
##  8 Germany 1963  life          70.1
##  9 Germany 1964  fertility      2.49
## 10 Germany 1964  life          70.7
## # ℹ 214 more rows
```

separate

The `separate` function is used to split a single column into multiple columns

- The default character to split on is an underscore `_`

```
data %>% separate(name, c("year", "variable_name"))
```

```
## # A tibble: 224 × 4
##    country year  variable_name value
##    <chr>   <chr> <chr>         <dbl>
##  1 Germany 1960  fertility      2.41
##  2 Germany 1960  life          69.3
##  3 Germany 1961  fertility      2.44
##  4 Germany 1961  life          69.8
##  5 Germany 1962  fertility      2.47
##  6 Germany 1962  life          70.0
##  7 Germany 1963  fertility      2.49
##  8 Germany 1963  life          70.1
##  9 Germany 1964  fertility      2.49
## 10 Germany 1964  life          70.7
## # ℹ 214 more rows
```

separate

The **separate** function is used to split a single column into multiple columns

- The default character to split on is an underscore _
- Adding **extra = "merge"** as an argument will ignore future instances of the splitting character

```
data %>% separate(name, c("year", "name"), extra = "merge")
```

```
## # A tibble: 224 × 4
##    country year  name            value
##    <chr>   <chr> <chr>           <dbl>
##  1 Germany 1960  fertility        2.41
##  2 Germany 1960  life_expectancy 69.3
##  3 Germany 1961  fertility        2.44
##  4 Germany 1961  life_expectancy 69.8
##  5 Germany 1962  fertility        2.47
##  6 Germany 1962  life_expectancy 70.0
##  7 Germany 1963  fertility        2.49
##  8 Germany 1963  life_expectancy 70.1
##  9 Germany 1964  fertility        2.49
## 10 Germany 1964  life_expectancy 70.7
## # ℹ 214 more rows
```

separate

The **separate** function is used to split a single column into multiple columns

- The default character to split on is an underscore _
- Adding **extra = "merge"** as an argument will ignore future instances of the splitting character

```
data %>%
    separate(name, c("year", "name"), extra = "merge") %>%
            pivot_wider()
```

```
## # A tibble: 112 × 4
##    country year  fertility life_expectancy
##    <chr>   <chr>     <dbl>           <dbl>
##  1 Germany 1960       2.41            69.3
##  2 Germany 1961       2.44            69.8
##  3 Germany 1962       2.47            70.0
##  4 Germany 1963       2.49            70.1
##  5 Germany 1964       2.49            70.7
##  6 Germany 1965       2.48            70.6
##  7 Germany 1966       2.44            70.8
##  8 Germany 1967       2.37            71.0
##  9 Germany 1968       2.28            70.6
## 10 Germany 1969       2.17            70.5
## # ℹ 102 more rows
```

unite

The **unite** function is used to combine two columns into one

```
var_names <- c("year", "first_variable_name", "second_variable_name")
data %>% separate(name, var_names, fill = "right")
```

```
## # A tibble: 224 × 5
##    country year  first_variable_name second_variable_name value
##    <chr>   <chr> <chr>               <chr>                 <dbl>
##  1 Germany 1960  fertility           <NA>                   2.41
##  2 Germany 1960  life                expectancy            69.3
##  3 Germany 1961  fertility           <NA>                   2.44
##  4 Germany 1961  life                expectancy            69.8
##  5 Germany 1962  fertility           <NA>                   2.47
##  6 Germany 1962  life                expectancy            70.0
##  7 Germany 1963  fertility           <NA>                   2.49
##  8 Germany 1963  life                expectancy            70.1
##  9 Germany 1964  fertility           <NA>                   2.49
## 10 Germany 1964  life                expectancy            70.7
## # ℹ 214 more rows
```

unite

The `unite` function is used to combine two columns into one

- The default character to unite on is an underscore _

```
data %>% separate(name, var_names, fill = "right") %>%
  unite(name, first_variable_name, second_variable_name)
```

```
## # A tibble: 224 × 4
##    country year  name             value
##    <chr>   <chr> <chr>            <dbl>
##  1 Germany 1960  fertility_NA      2.41
##  2 Germany 1960  life_expectancy 69.3
##  3 Germany 1961  fertility_NA      2.44
##  4 Germany 1961  life_expectancy 69.8
##  5 Germany 1962  fertility_NA      2.47
##  6 Germany 1962  life_expectancy 70.0
##  7 Germany 1963  fertility_NA      2.49
##  8 Germany 1963  life_expectancy 70.1
##  9 Germany 1964  fertility_NA      2.49
## 10 Germany 1964  life_expectancy 70.7
## # ℹ 214 more rows
```

unite

The `unite` function is used to combine two columns into one

- The default character to unite on is an underscore _

```
data %>% separate(name, var_names, fill = "right") %>%
  unite(name, first_variable_name, second_variable_name) %>%
  pivot_wider()
```

```
## # A tibble: 112 × 4
##    country year  fertility_NA life_expectancy
##    <chr>   <chr>        <dbl>           <dbl>
##  1 Germany 1960          2.41            69.3
##  2 Germany 1961          2.44            69.8
##  3 Germany 1962          2.47            70.0
##  4 Germany 1963          2.49            70.1
##  5 Germany 1964          2.49            70.7
##  6 Germany 1965          2.48            70.6
##  7 Germany 1966          2.44            70.8
##  8 Germany 1967          2.37            71.0
##  9 Germany 1968          2.28            70.6
## 10 Germany 1969          2.17            70.5
## # ℹ 102 more rows
```

unite

The `unite` function is used to combine two columns into one

- The default character to unite on is an underscore _

```
data %>% separate(name, var_names, fill = "right") %>%
  unite(name, first_variable_name, second_variable_name) %>%
  pivot_wider() %>%
  rename(fertility = fertility_NA)
```

```
## # A tibble: 112 × 4
##    country year  fertility life_expectancy
##    <chr>   <chr>     <dbl>           <dbl>
##  1 Germany 1960       2.41            69.3
##  2 Germany 1961       2.44            69.8
##  3 Germany 1962       2.47            70.0
##  4 Germany 1963       2.49            70.1
##  5 Germany 1964       2.49            70.7
##  6 Germany 1965       2.48            70.6
##  7 Germany 1966       2.44            70.8
##  8 Germany 1967       2.37            71.0
##  9 Germany 1968       2.28            70.6
## 10 Germany 1969       2.17            70.5
## # ℹ 102 more rows
```

Summary

- The `pivot_longer` function converts wide data into tidy data
- The `pivot_wider` function converts tidy data into long data
- The `separate` function splits a single column into multiple columns
- The `unite` function combines two columns into a single column